PyCallingAgent 演讲稿（中文要点版）

> 配套 PPT：`pycallingagent_technical_deck.pptx`（共 10 页） 建议总时长：10–12 分钟。每页 1–1.5 分钟。

Slide 1 · 封面 — PyCallingAgent

一句话定位：这是一个面向金融研究场景的 Agent 产品，底层跑在一个持久化的 Python 运行时上。

讲述要点： - 各位好，今天分享的项目叫 **PyCallingAgent**。 - 它不是一个全新的 Agent 框架，而是一个**产品层的工作**：底层的运行时与 Skills 机制来自 CaveAgent 的工作，我们在它之上做了金融研究向的产品化封装。 - 产品形态很简单：**一个输入框 + 一页结果**。用户给一个自然语言 prompt，系统返回文字回答，以及图表、表格、可下载的报告这样的"工作产物"。 - 它已经部署在公开 URL `pycallingagent.onrender.com`，任何人都能点进去用。 - 强调一下：**仅用于研究演示，不构成任何投资建议**。

Slide 2 · 问题与动机 — 传统 Function Calling 的文本瓶颈

核心观点：数据密集的研究任务，用 JSON function calling 很吃亏。

讲述要点： - 现在主流的 LLM 工具调用范式是 **JSON function calling**：模型吐一段 JSON，外部去执行，再把结果变成字符串喂回去。 - 对一般聊天够用，但对研究类任务有三个硬痛点： 1. **序列化税**：DataFrame、图、模型，任何复杂对象都得先被压成字符串才能让 LLM "看到"。 2. **无持久状态**：每次调用都是全新的一次，变量、连接、会话都活不过一轮。 3. **组合性弱**：模型没法用几行

代码把内存里的对象串起来，只能一遍遍用文字复述数据。 - 合起来就是一句话：**我们其实想让 LLM 直接驱动一个活着的 Python 进程，而不是让它反复"复述"这个进程的字符串快照**。 - 这就是我们整个设计的出发点。

Slide 3 · 设计哲学 — 从文本生成器到进程操作者

核心观点：我们把 PyCallingAgent 放在工具调用范式演进的 Gen 3。

讲述要点： - 这张图展示了工具调用范式的四代演进： - **Gen 0**：纯文本，没有工具。 - **Gen 1**：JSON 函数调用，无状态，纯字符串输入输出。 - **Gen 2**：Code-as-action，每次起一个临时解释器，执行后依然以字符串回传。 - **Gen 3**：**持久化运行时** —— 内核长驻，对象跨轮次存活。 - PyCallingAgent 的底层定位在 Gen 3。 - 要让 Gen 3 真正能跑起来，有三个支柱： 1. **双轨循环**：把 LLM 的推理历史和 Python 的执行状态分开，但保持同步。 2. **可确定性状态访问**：运行时是可检查的，我们能读出每一个变量，给自动化评测和奖励信号留了口子。 3. **运行时感知 Skills**：Skills 不仅仅是文字说明，而是真的把可调用对象注入到运行时里。

Slide 4 · 核心架构 — 语义轨 + 运行时轨

核心观点：一次 agent loop = 语义轨迭代 + 运行时轨状态更新，两者通过"观察"耦合。

讲述要点： - 上半部分是 **语义轨**：也就是 LLM 看到的内容。更新规则 `h_t = LLM(x_t, h_{t-1})`，就是大家熟悉的对话历史滚动。 - 下半部分是 **运行时轨**：一个长驻的 IPython 内核。更新规则 `s_t = exec(c_t, s_{t-1})`，代码块在同一个 Python 命名空间里依次执行。 - 中间的橙色箭头代表"观察整形"（observation shaping）： - 运行时里完整的对象（比如一个上千行的 DataFrame）**不会**整个塞回 prompt。 - 只有简短的摘要（shape、列名、头几行等）会回到 context。 - 这带来一个很重要的性质：**LLM 的 context 很干净，真正的工作空间是运行时**。 - 语

境开销和工作容量被解耦了——context window 不再是瓶颈。

Slide 5 · 运行时 + Skills — 注入 API 与懒加载

核心观点：三种 Runtime 原语 + 懒加载 Skills，让 LLM 用"很少 token"看到"很多能力"。

讲述要点： - 左边的代码块展示了注入 API： - 新建一个 `IPythonRuntime()`； - 把真实的 `pandas.DataFrame` 用 `Variable` 注入； - 把 Python 函数用 `Function` 注入； - 然后 `CaveAgent(model, runtime).run(prompt)` 就能跑。 - 注入的三种原语： - **Variable**：任何 Python 对象，按名字可以取回。 - **Function**：一个带 docstring 的可调用对象。 - **Type**：自动从 Pydantic / dataclass / Enum 抽 schema。 - 右下角是 **Skill 机制**，这是非常关键的一点： - 启动时 LLM 只看到每个 skill 的元数据（大约 100 tokens）。 - 只有当它调用 `activate_skill(name)` 时，完整指令和可调用对象才会被真正注入。 - 这是"按需加载"，在 context 预算上非常友好。 - 一句话总结：**Context 登记的是"目录"，真正的"工具库"长在运行时里**。

Slide 6 · 金融研究工作流 — 按下 Send 之后发生了什么

核心观点：一次 prompt 触发一整条 pipeline，输出既包含文字答案，也包含可视化产物。

讲述要点： - 整条 pipeline 分 5 步： 1. **解析 prompt**：路由到金融管线还是通用聊天；抽出 ticker、ETF 或市场代理。 2. **拉取有据可查的数据**：SEC EDGAR 做基本面；Alpha Vantage 可选；如果外部 API 不可用，就走 public-data 回落模式。 3. **注入到运行时**：价格 DataFrame、基本面数据、辅助函数全部变成 runtime 中的活变量。 4. **跑 Agent 主循环**：LLM 直接对真实对象调用 skill，不再靠"凭记忆描述数字"。 5. **物化输出**：图表、表格、Markdown 报告、快照卡片保存到本次 session。 - 强调一下和 dashboard 的差别：**一轮对话既有语言输出，也有看得见的工作产物**，

这是它"像 agent"而不"像报表"的原因。 - 另外这里也说明了产品层的一个小优化：**非金融 prompt（比如"hello"）会旁路整条金融管线，直接走通用 LLM QA**，不会因为缺 ticker 就报错。

Slide 7 · 应用层适配 — 把 Agent 包成 FastAPI 产品

核心观点：Agent 内核不变，应用层加了会话/运行的编排 + 实时更新 + 产物渲染。

讲述要点： - 上方的一排四个框是服务架构： - **Browser UI**：Jinja2 + 原生 JS，负责输入框、聊天、产物面板。 - **FastAPI 服务**：会话、运行、SSE / 轮询、文件上传、每个产物的下载端点。 - **Agent Runner**（`FinancialResearchRunner`）：推断 ticker → 拉数据 → 跑 agent → 存产物。 - **Data Layer**：SEC EDGAR + 可选 Alpha Vantage + public-data 回落。 - 下面三个细节卡强调的是产品层面的工程化： 1. **Session + Run 编排**：每个 session 有独立的消息历史和工作目录；每次 prompt 是一个可取消的 run；运行时内核可以跨轮复用。 2. **页面实时更新**：事件流用 SSE 推，弱网下回退成轮询；前端维护一个"最近事件滑动窗口"作为 Live Status。 3. **产物渲染**：表格和图表在页面内直接预览；完整文件可下载；Markdown 报告在页面内渲染；非金融问题直接走聊天通道。 - 一句话：**底层是科研级 runtime，外层是产品级 Web 应用**。

Slide 8 · 部署 — 从 GitHub 到公开 URL

核心观点：部署路径故意保持极小，避免演示翻车。

讲述要点： - 流程就三步： 1. **GitHub** 是 source of truth。推 `main` 分支，Render 自动监听并重新部署。 2. **Render Web Service** 负责构建和启动： - Build：`pip install ".[openai,webapp]"` - Start：`python -m uvicorn cave_agent.webapp.app:create_app --factory` - Render 直接分配一个公开 URL，不需要自己配 DNS。 3. **Runtime config** 是一小撮环境变量： - `LLM_MODEL_ID` / `LLM_API_KEY` / `LLM_BASE_URL` 负责模型接入； - `SEC_USER_AGENT` 是

SEC EDGAR 要求的 UA； - `ALPHAVANTAGE_API_KEY` 是可选的，没有也能跑，走 public-data 回落。 - 最后强调公开 URL：`pycallingagent.onrender.com`，老师或同学可以直接点进去试。 - 为什么要留 public-data 回落？**免费层 API 随时可能限速，我们不希望演示因为第三方抽风而挂掉**。

Slide 9 · Demo 1 — Prompt 到研究产物

建议演示 prompt：

> "Summarize Apple and show the recent price trend."

讲述要点： - 现场打开公开 URL，在输入框粘贴这条 prompt。 - 重点让观众看到： - 页面右侧的 **Live Status** 在流式更新（抓数据、跑 agent、保存产物）； - 最终页面上同时出现：**文字摘要 + 价格趋势图 + 数据表 + 可下载报告**。 - 口播一句：**"它不是把 Apple 的数据背给我看，它是现场算出来的。"**

Slide 10 · Demo 2 — 对比和产物

建议演示 prompt：

> "Compare AMD and Nvidia on valuation and performance."

讲述要点： - 演示跨标的对比的场景。 - 重点展示： - **比较表格**：估值和业绩指标并排； - **多序列图表**：两家股价或基本面趋势叠在一张图上； - **Markdown 报告**：页面内渲染，点一下就能下载。 - 最后落点：**这就是所谓 "agent-like" 体验——一个 prompt 进去，拿到一份小型研究简报出来。** - 谢谢大家，欢迎提问。

额外的口播小贴士

- **开场**（Slide 1）和**落幕**（Slide 10）各保留 20 秒做"连接"，别一上来就讲技术。
- **Slide 2–5** 是技术核心，每页讲 1 分钟左右即可，别在细节里陷太久。
- **Slide 6–8** 是产品/工程部分，语气可以更轻，多讲"为什么这么做"，少讲"怎么做"。
- **Slide 9–10** 是现场演示，建议先讲完要做什么、**再**打开浏览器，避免在等加载时冷场。
- 如果 Q&A 里被问到"这和 CaveAgent 的关系"，老老实实答：**CaveAgent 提供 runtime 和 skills 作为 agent 核；PyCallingAgent 在它之上做金融研究产品和 Web 层适配**。